

Data Structures & Algorithms

A comprehensive reference covering every major concept,
from Big O notation to dynamic programming.

Contents

Big O Notation · Arrays · Linked Lists · Stacks & Queues
Hash Tables · Trees & Heaps · Graphs
Searching · Sorting · Dynamic Programming · Cheat Sheet

Contents

1	Big O Notation	3
1.1	Growth Rate Hierarchy	3
1.2	Common Complexities Reference	3
1.3	Rules for Simplification	4
2	Arrays	4
2.1	Complexity	4
2.2	Memory Layout	4
2.3	Dynamic Arrays	4
2.4	Key Patterns	4
3	Linked Lists	5
3.1	Variants	5
3.2	Complexity	5
3.3	Common Techniques	5
4	Stacks and Queues	6
4.1	Stack — LIFO	6
4.2	Queue — FIFO	6
4.3	Monotonic Stack	6
5	Hash Tables	6
5.1	Complexity	6
5.2	Collision Resolution	7
6	Trees and Heaps	7
6.1	Binary Search Tree (BST)	7
6.2	BST Operations Complexity	7
6.3	Tree Traversals	8
6.4	Self-Balancing Trees	8
6.5	Heaps	8
7	Graphs	8
7.1	Key Terminology	8
7.2	Representations	9
7.3	Graph Traversals	9
7.3.1	Breadth-First Search (BFS) — $O(V + E)$	9
7.3.2	Depth-First Search (DFS) — $O(V + E)$	9
7.4	Key Graph Algorithms	10
8	Searching Algorithms	10
8.1	Linear Search — $O(n)$	10
8.2	Binary Search — $O(\log n)$	10
9	Sorting Algorithms	11
9.1	Complexity Summary	11
9.2	Algorithm Descriptions	11
10	Dynamic Programming	12
10.1	Two Approaches	12
10.2	Example: Fibonacci	12

10.3 DP Table — Fibonacci	12
10.4 Classic DP Problems	12
11 Cheat Sheet	13
11.1 Data Structure Complexities	13
11.2 When to Use Each Structure	13
11.3 Problem-Solving Patterns	14

Big O Notation

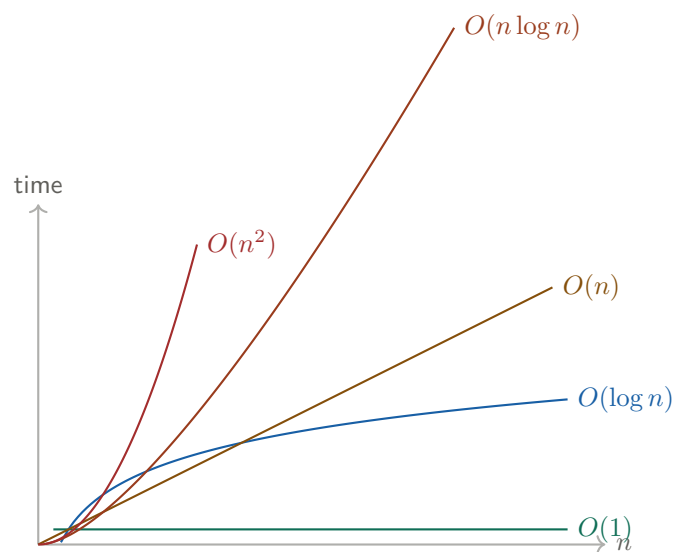
Big O notation describes how the **time** or **space** an algorithm consumes grows as the input size n increases. It focuses on the *dominant* growth term and ignores constants, giving a machine-independent upper bound on complexity.

Formal definition. We say $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. In plain terms: beyond some threshold, $g(n)$ is an upper bound on $f(n)$, up to a constant factor.

Growth Rate Hierarchy

From fastest (best) to slowest (worst) as $n \rightarrow \infty$:

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(2^n) \subset O(n!)$$



Common Complexities Reference

Notation	Name	Rating	Typical examples
$O(1)$	Constant	Excellent	Array index access, hash table lookup
$O(\log n)$	Logarithmic	Great	Binary search, balanced BST operations
$O(n)$	Linear	Good	Linear scan, single-pass traversal
$O(n \log n)$	Linearithmic	Fair	Merge sort, heap sort, FFT
$O(n^2)$	Quadratic	Slow	Bubble/selection sort, naive string match
$O(2^n)$	Exponential	Terrible	Naive Fibonacci, power set enumeration
$O(n!)$	Factorial	Terrible	Brute-force travelling salesman

Table 1: Common complexity classes ordered by growth rate

Always evaluate **both** time and space complexity. An $O(1)$ time solution that requires $O(n)$ space may be worse than an $O(n)$ time, $O(1)$ space solution depending on your constraints.

Rules for Simplification

- **Drop constants:** $O(3n) = O(n)$
- **Drop lower-order terms:** $O(n^2 + n) = O(n^2)$
- **Different inputs, different variables:** Two separate arrays of size m and n give $O(m + n)$, not $O(n^2)$
- **Nested loops multiply:** A loop of n inside a loop of m gives $O(mn)$

Arrays

An array is a **contiguous block of memory** holding elements of the same type (in typed languages). Elements are accessed directly by index in $O(1)$ — the CPU computes the address as $\text{base} + i \times \text{element_size}$.

Complexity

Operation	Complexity	Notes
Access by index	$O(1)$	Direct memory calculation
Search (unsorted)	$O(n)$	Must scan all elements
Search (sorted)	$O(\log n)$	Binary search
Insert at end	$O(1)$ amortized	Dynamic resizing
Insert at middle	$O(n)$	Elements must shift
Delete at middle	$O(n)$	Elements must shift

Memory Layout



Dynamic Arrays

Dynamic arrays (Python `list`, JavaScript `Array`, C++ `vector`) resize automatically. When the array is full, a new block of *double* the current capacity is allocated and all elements are copied — an $O(n)$ operation. Because this happens rarely, the **amortised** cost of appending is $O(1)$.

Key Patterns

Two-pointer technique — for sorted arrays, maintain a left pointer at index 0 and a right pointer at the last index. Move them inward based on the sum compared to the target. Finds a pair summing to a target in $O(n)$ instead of $O(n^2)$:

Listing 1: Two-pointer: find pair summing to target

```

1 def two_sum_sorted(arr, target):
2     l, r = 0, len(arr) - 1
3     while l < r:
4         s = arr[l] + arr[r]
5         if s == target:
6             return (l, r)

```

```

7     elif s < target:
8         l += 1
9     else:
10        r -= 1
11    return None

```

Sliding window — maintain a window $[l, r]$ that slides over the array to solve subarray problems in $O(n)$.

Linked Lists

A linked list is a chain of **nodes**, each holding a value and one or more pointers to neighbouring nodes. Memory is not contiguous — each node may be anywhere in the heap.

Variants

Singly linked



Doubly linked



Complexity

Operation	Complexity	Notes
Access by index	$O(n)$	Must traverse from head
Search	$O(n)$	Sequential scan
Insert (with ptr)	$O(1)$	Rewire next/prev pointers only
Delete (with ptr)	$O(1)$	Rewire next/prev pointers only
Insert at head	$O(1)$	Update head pointer

Insertion and deletion are $O(1)$ only if you already hold a pointer to the relevant node. Finding that node still takes $O(n)$.

Common Techniques

- **Fast/slow pointers (Floyd's cycle detection):** move two pointers at different speeds to detect cycles in $O(n)$ time and $O(1)$ space.
- **Reverse a linked list:** iterate once, rewiring each node's next to the previous node.
- **Find the middle:** advance slow by 1 and fast by 2 simultaneously; when fast reaches the end, slow is at the middle.

Listing 2: Reverse a singly linked list — $O(n)$ time, $O(1)$ space

```

1 def reverse(head):
2     prev, curr = None, head
3     while curr:
4         nxt = curr.next

```

```

5     curr.next = prev
6     prev = curr
7     curr = nxt
8     return prev

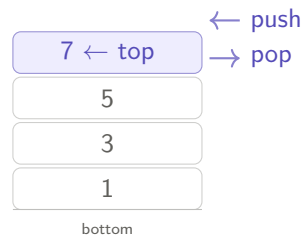
```

Stacks and Queues

Abstract data types that **restrict access** to elements. Both support $O(1)$ insertion and deletion at their designated ends.

Stack — LIFO

Last In, First Out. Elements are added and removed from the same end (the *top*).



Use cases: function call stack, undo/redo, expression evaluation, DFS, balanced bracket checking.

Balanced bracket checker:

Listing 3: Check balanced brackets with a stack

```

1 def is_balanced(s):
2     stack = []
3     pairs = {'(': ')', '{': '}', '[': ']'}
4     for ch in s:
5         if ch in pairs:
6             stack.append(ch)
7         elif ch in pairs.values():
8             if not stack or pairs[stack.pop()] != ch:
9                 return False
10    return len(stack) == 0

```

Queue — FIFO

First In, First Out. Elements are added at the back (*enqueue*) and removed from the front (*dequeue*).

Use cases: BFS, task scheduling, print queue, message queues, rate limiting.

Monotonic Stack

A stack that maintains elements in strictly increasing or decreasing order. Useful for problems involving the *next greater element* pattern, solved in $O(n)$.

Hash Tables

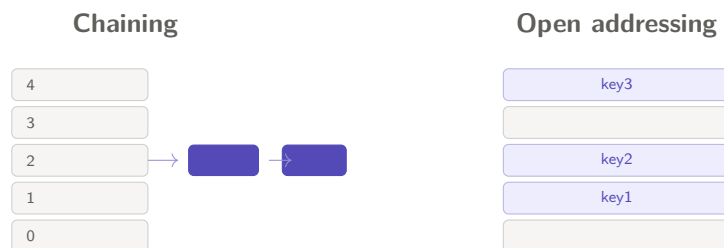
A hash table maps **keys to values** using a hash function $h(k) \rightarrow [0, M)$ where M is the number of buckets. Provides $O(1)$ average-case lookup, insertion, and deletion.

Complexity

Worst case occurs when all keys hash to the same bucket (degenerate case with a poor hash function or adversarial input).

Operation	Average	Worst
Lookup	$O(1)$	$O(n)$
Insert	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

Collision Resolution



- **Chaining:** each bucket holds a linked list; multiple keys may share a bucket.
- **Open addressing:** on collision, probe for the next empty slot (linear, quadratic, or double hashing). Cache-friendly; no pointer overhead.

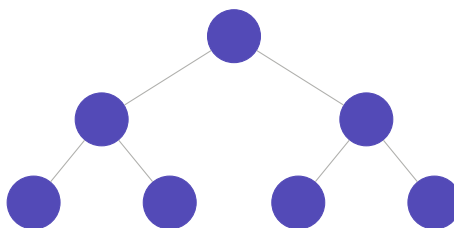
Load factor $\lambda = n/M$ (items / buckets). When $\lambda > 0.7$, the table is rehashed into a larger array ($O(n)$ one-time cost). Python dicts target $\lambda \leq 0.67$. Java's HashMap rehashes at 0.75.

Trees and Heaps

A **tree** is a connected, acyclic graph with a designated *root*. Every non-root node has exactly one parent. Trees model hierarchical data: file systems, HTML DOM, company org-charts, decision trees.

Binary Search Tree (BST)

A binary tree where every node satisfies: *left subtree values* < *node value* < *right subtree values*. This ordering enables efficient search.



BST Operations Complexity

Operation	Average (balanced)	Worst (degenerate)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

A *degenerate* BST (inserting already-sorted data) degrades to a linked list, giving $O(n)$ per operation.

Tree Traversals

Traversal	Order	Use case
In-order	Left $\rightarrow$ Root $\rightarrow$ Right	Produces sorted output from BST
Pre-order	Root $\rightarrow$ Left $\rightarrow$ Right	Copy / serialize a tree
Post-order	Left $\rightarrow$ Right $\rightarrow$ Root	Delete a tree; evaluate expression trees
Level-order	Row by row (BFS)	Shortest path; level-by-level processing

Self-Balancing Trees

Self-balancing trees guarantee $O(\log n)$ height at all times, preventing the degenerate case:

- **AVL tree:** maintains the balance factor $|\text{height}(L) - \text{height}(R)| \leq 1$ at every node using rotations. Strictly balanced — ideal for lookup-heavy workloads.
- **Red-Black tree:** nodes are coloured red or black with invariants that bound height to $2 \log_2(n+1)$. Fewer rotations than AVL — preferred for insert/delete-heavy workloads (used in Linux scheduler, Java's `TreeMap`, C++ `std::map`).
- **B-Tree:** generalises BSTs to nodes with many keys. $O(\log n)$ I/O operations — the standard structure for database indexes and file systems.

Heaps

A **heap** is a *complete* binary tree satisfying the heap property:

- **Min-heap:** every parent $\leq$ its children. The root is always the minimum.
- **Max-heap:** every parent $\geq$ its children. The root is always the maximum.

Heaps are stored compactly in an array: for a node at index i , children are at $2i + 1$ and $2i + 2$; the parent is at $\lfloor (i - 1) / 2 \rfloor$.

Operation	Complexity
Get min/max	$O(1)$
Insert	$O(\log n)$
Extract min/max	$O(\log n)$
Build from array	$O(n)$

Python's `heapq` module provides a min-heap. Use `heapq.heappush` and `heapq.heappop`. For a max-heap, negate values on insertion. Heaps are the canonical implementation of a **priority queue**.

Graphs

A graph $G = (V, E)$ consists of a set of **vertices** V and a set of **edges** E . Trees are a special case of graphs (connected, acyclic). Graphs model networks, maps, dependencies, social connections, state machines, and more.

Key Terminology

- **Directed / Undirected:** edges have or lack a direction.

- **Weighted / Unweighted:** edges have or lack a numeric cost.
- **Cyclic / Acyclic:** contains or lacks cycles. A Directed Acyclic Graph (DAG) is used for dependency resolution and scheduling.
- **Connected:** a path exists between every pair of vertices.
- **Degree:** number of edges incident to a vertex.

Representations

Representation	Space	Edge lookup	Best for
Adjacency list	$O(V + E)$	$O(\deg(V))$	Sparse graphs (most real-world graphs)
Adjacency matrix	$O(V^2)$	$O(1)$	Dense graphs, quick edge existence check
Edge list	$O(E)$	$O(E)$	Sorting edges (Kruskal's algorithm)

Graph Traversals

Breadth-First Search (BFS) — $O(V + E)$

Explores all neighbours of a vertex before moving deeper. Uses a **queue**.

Listing 4: BFS — finds shortest path in unweighted graph

```

1 from collections import deque
2
3 def bfs(graph, start):
4     visited = {start}
5     queue = deque([start])
6     order = []
7     while queue:
8         node = queue.popleft()
9         order.append(node)
10        for neighbour in graph[node]:
11            if neighbour not in visited:
12                visited.add(neighbour)
13                queue.append(neighbour)
14    return order

```

Depth-First Search (DFS) — $O(V + E)$

Explores as deep as possible before backtracking. Uses a **stack** (or recursion).

Listing 5: DFS — cycle detection, topological sort

```

1 def dfs(graph, start, visited=None):
2     if visited is None:
3         visited = set()
4     visited.add(start)
5     for neighbour in graph[start]:
6         if neighbour not in visited:
7             dfs(graph, neighbour, visited)
8     return visited

```

Algorithm	Complexity	Purpose
BFS	$O(V + E)$	Shortest path (unweighted)
DFS	$O(V + E)$	Cycle detection, topological sort, SCCs
Dijkstra's	$O((V + E) \log V)$	Shortest path (non-negative weights)
Bellman-Ford	$O(VE)$	Shortest path (negative weights)
Floyd-Warshall	$O(V^3)$	All-pairs shortest paths
Kruskal's / Prim's	$O(E \log E)$	Minimum spanning tree
Topological sort	$O(V + E)$	Dependency ordering in DAGs

Key Graph Algorithms

Searching Algorithms

Linear Search — $O(n)$

Examines each element sequentially. Works on any array, sorted or not.

Listing 6: Linear search

```

1 def linear_search(arr, target):
2     for i, val in enumerate(arr):
3         if val == target:
4             return i    #  $O(i)$  comparisons
5     return -1

```

Binary Search — $O(\log n)$

Requires a **sorted** array. Repeatedly halves the search space by comparing the target to the middle element.

searching for 55 — mid=35, go right



Listing 7: Binary search — iterative

```

1 def binary_search(arr, target):
2     l, r = 0, len(arr) - 1
3     while l <= r:
4         mid = (l + r) // 2
5         if arr[mid] == target:
6             return mid
7         elif arr[mid] < target:
8             l = mid + 1
9         else:
10            r = mid - 1
11    return -1

```

$\log_2(10^9) \approx 30$. Binary search finds an element in one billion sorted values with *at most 30 comparisons*. This is the practical power of $O(\log n)$.

Sorting Algorithms

Complexity Summary

Algorithm	Best	Average	Worst	Space	Stable?
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Yes
Radix sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + k)$	Yes

Table 2: k = range of input values (counting/radix sort)

Algorithm Descriptions

Bubble sort: repeatedly swap adjacent elements that are out of order. Simple, but $O(n^2)$ in practice. Useful for nearly-sorted data (best case $O(n)$).

Insertion sort: for each element, insert it into its correct position within the already-sorted left portion. Efficient for small or nearly-sorted arrays — used as the base case in Timsort.

Merge sort: divide the array in half, recursively sort each half, then merge the two sorted halves. Guaranteed $O(n \log n)$ and stable — used in Python's `sorted()` and Java's `Arrays.sort` for objects.

Listing 8: Merge sort

```

1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = merge_sort(arr[:mid])
6     right = merge_sort(arr[mid:])
7     return merge(left, right)
8
9 def merge(left, right):
10    result = []
11    i = j = 0
12    while i < len(left) and j < len(right):
13        if left[i] <= right[j]:
14            result.append(left[i]); i += 1
15        else:
16            result.append(right[j]); j += 1
17    return result + left[i:] + right[j:]

```

Quick sort: choose a pivot, partition elements into $<$ pivot and $>$ pivot, then recursively sort each partition. $O(n \log n)$ on average but $O(n^2)$ with bad pivot choice (mitigated by random pivot selection). The fastest in practice due to excellent cache performance.

Stability: a sort is *stable* if equal elements maintain their original relative order. This matters when sorting objects by one field after already sorting by another field.

Dynamic Programming

Dynamic programming (DP) solves problems with **overlapping subproblems** and **optimal substructure** by storing results of subproblems to avoid redundant computation.

Two conditions for DP

1. **Optimal substructure:** the optimal solution to the problem contains optimal solutions to its subproblems.
2. **Overlapping subproblems:** the same subproblems are solved multiple times in a naive recursive approach.

Two Approaches

- **Top-down (memoization):** write the natural recursive solution, then cache results in a hash map or array. Intuitive to write; uses the call stack.
- **Bottom-up (tabulation):** fill a table starting from the smallest subproblem up to the answer. Avoids recursion overhead and stack overflow risk; usually faster in practice.

Example: Fibonacci

Naive recursion — $O(2^n)$ time: each call branches into two, forming a tree of exponential size.

Memoization — $O(n)$ time, $O(n)$ space:

Listing 9: Fibonacci with memoization

```
1 from functools import lru_cache
2
3 @lru_cache(maxsize=None)
4 def fib(n):
5     if n <= 1:
6         return n
7     return fib(n-1) + fib(n-2)
```

Tabulation — $O(n)$ time, $O(1)$ space (space-optimised):

Listing 10: Fibonacci with tabulation — $O(1)$ space

```
1 def fib(n):
2     a, b = 0, 1
3     for _ in range(n):
4         a, b = b, a + b
5     return a
```

DP Table — Fibonacci

$\leftarrow \text{fib}(9) = 34$

0	1	1	2	3	5	8	13	21	34
0	1	2	3	4	5	6	7	8	9

Classic DP Problems

Problem	Complexity	Key idea
Fibonacci	$O(n)$	1D table, two previous values
Coin change	$O(n \cdot k)$	Min coins to reach amount n with k coin types
Knapsack 0/1	$O(n \cdot W)$	2D table indexed by item count and weight
Longest Common Subsequence	$O(mn)$	2D grid on two strings
Longest Increasing Subsequence	$O(n \log n)$	Patience sorting with binary search
Edit distance	$O(mn)$	Insert / delete / substitute between two strings
Matrix chain multiplication	$O(n^3)$	Optimal parenthesisation

Cheat Sheet

Data Structure Complexities

Structure	Access	Search	Insert	Delete
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Hash Table	—	$O(1)^\dagger$	$O(1)^\dagger$	$O(1)^\dagger$
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap	$O(1)$ top	$O(n)$	$O(\log n)$	$O(\log n)$

Table 3: * given pointer to node; † average case

When to Use Each Structure

Requirement	Use
Fast random access by index	Array
Frequent insertions / deletions	Linked list
Key-to-value mapping, fast lookup	Hash table
LIFO access, undo, call stack	Stack
FIFO access, BFS, scheduling	Queue
Always retrieve min or max fast	Min/Max Heap
Sorted data, range queries	Balanced BST (AVL, Red-Black)
Network / relationship modelling	Graph
Overlapping subproblems	Dynamic programming
Prefix lookup, autocomplete	Trie

Problem-Solving Patterns

1. **Sliding window** — fixed or variable-size window sliding over an array. Solves subarray/substring problems in $O(n)$.
2. **Two pointers** — one pointer from each end; move inward based on a condition. Replaces $O(n^2)$ nested loops with $O(n)$.
3. **Fast & slow pointers** — Floyd's cycle detection; finding the middle of a linked list.
4. **Merge intervals** — sort by start, then greedily merge overlapping intervals.
5. **Binary search on answer** — when feasibility is monotonic, binary search on the answer space rather than the array.
6. **BFS for shortest path** — in unweighted graphs, BFS always finds the shortest path.
7. **DFS + backtracking** — generate all combinations/permutations; prune early.
8. **Topological sort** — course prerequisites, build order, any DAG dependency problem.
9. **Union-Find** — efficiently check if two nodes are connected; detect cycles in undirected graphs.
10. **Monotonic stack/queue** — next greater/smaller element; sliding window maximum.

Interview strategy: (1) Clarify constraints and examples. (2) Think out loud about brute force first, then optimise. (3) Identify the pattern (array? graph? DP?). (4) Write clean code with meaningful variable names. (5) Test edge cases: empty input, single element, all duplicates.